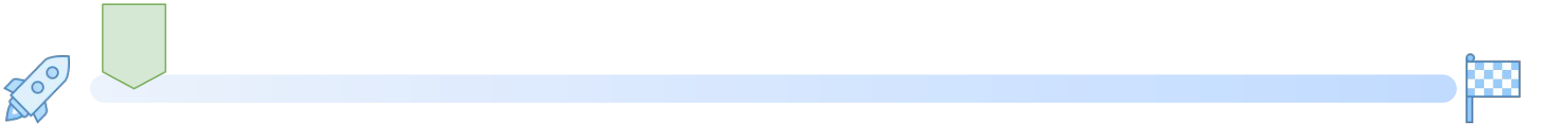
OpenTelemetry

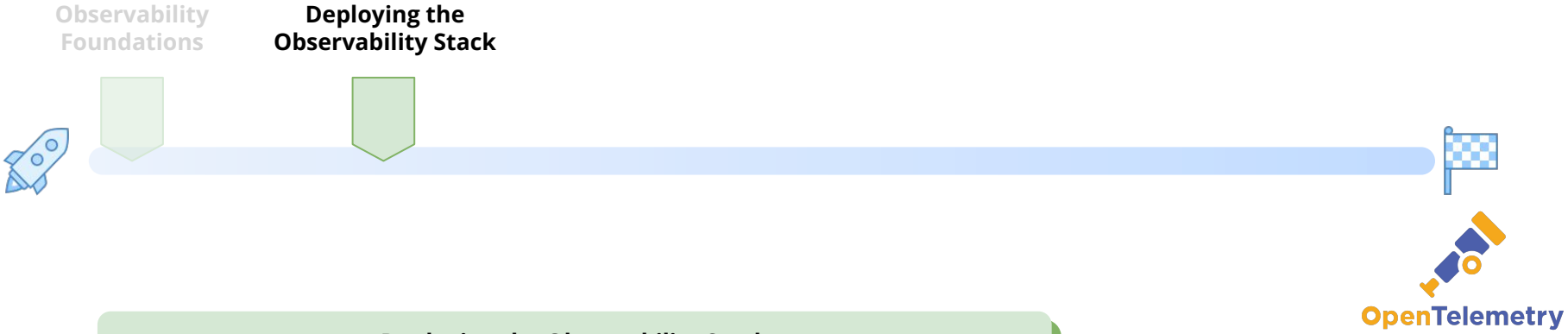
Observability Foundations



OpenTelemetry

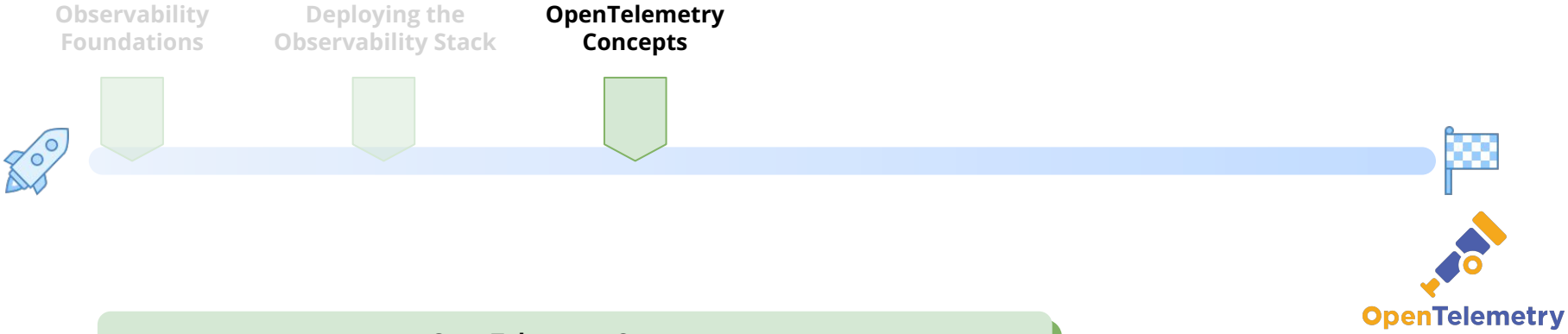
Observability Foundations

- ✅ Metrics, logs, and traces as the three pillars of observability
- ✅ SLIs, SLOs, SLAs, and error budgets for reliability management
- ✅ The Four Golden Signals as a universal monitoring baseline



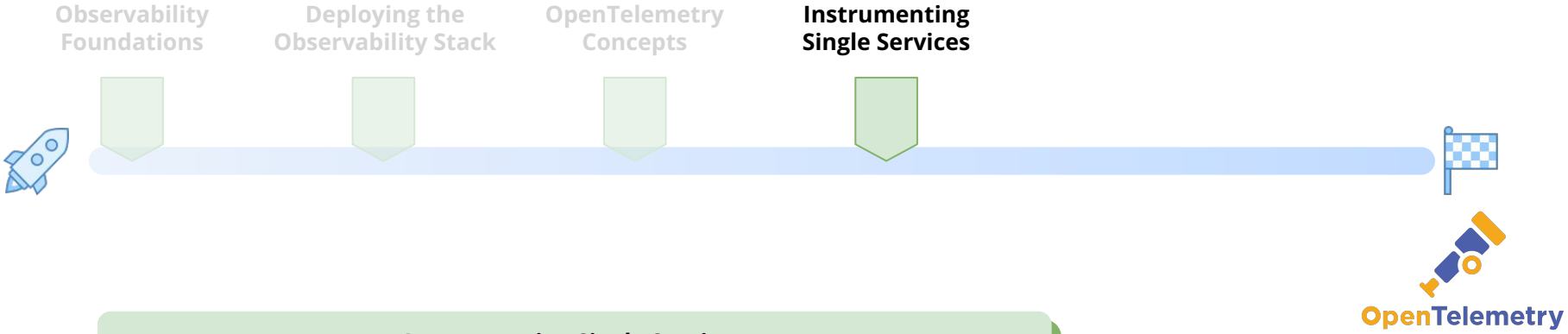
Deploying the Observability Stack

- ✅ Prometheus, Loki, and Tempo as backends for all three signal types
- ✅ Grafana as the unified visualization layer across all signals
- ✅ A distributed sample application as the instrumentation target



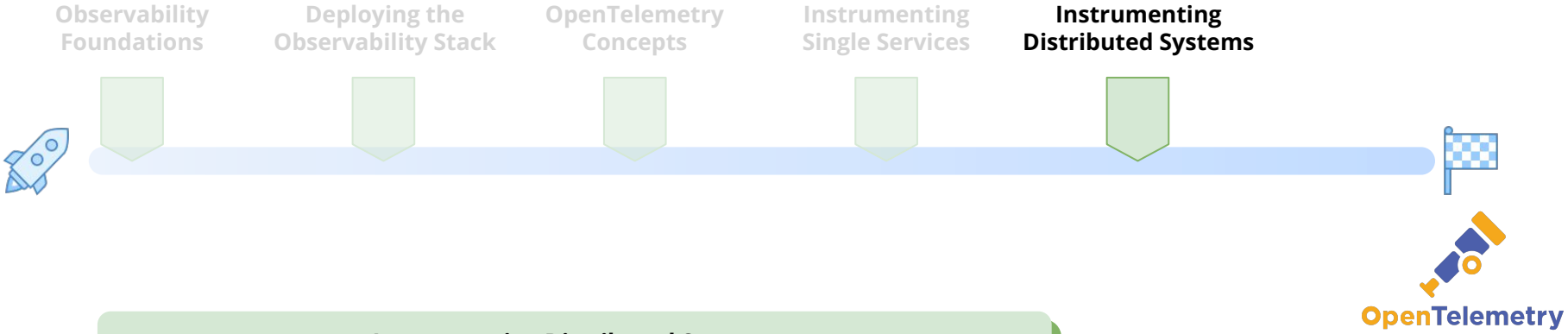
OpenTelemetry Concepts

- ✅ Vendor-neutral telemetry with OTel APIs, SDKs, and the Collector
- ✅ Automatic and manual instrumentation approaches
- ✅ Semantic conventions for consistent, interoperable telemetry



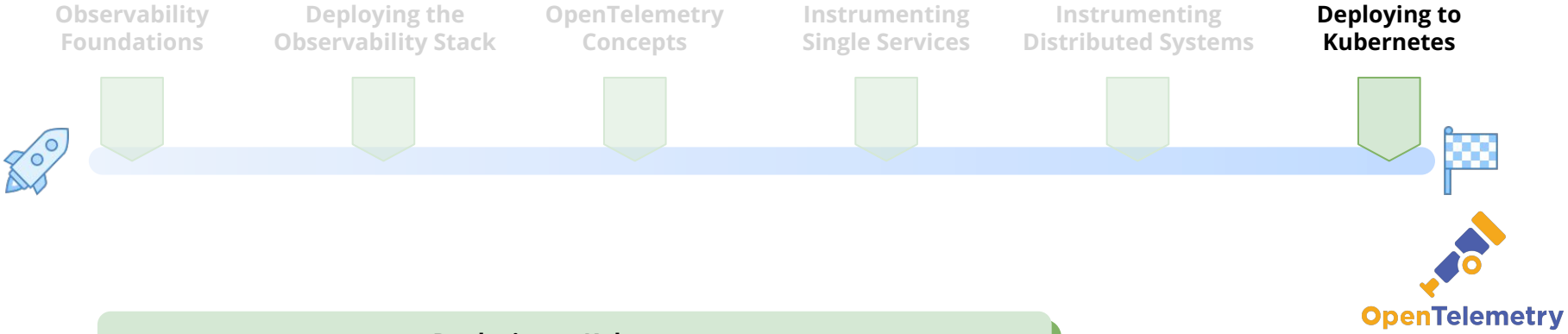
Instrumenting Single Services

- ✅ The OTel Collector as a centralized, vendor-agnostic telemetry pipeline
- ✅ Metrics, traces, and logs instrumentation for the frontend service
- ✅ Log-trace correlation in Grafana and environment-driven configuration



Instrumenting Distributed Systems

- ✅ **Full signal coverage for the Python worker service**
- ✅ **Distributed context propagation across asynchronous service boundaries**
- ✅ **Custom metrics, spans, Redis exporter, and structured logging**



Deploying to Kubernetes

- ✅ **Kubernetes manifests for the application and observability stack**
- ✅ **Configuration management with Kustomize across multiple namespaces**
- ✅ **Deploying to a live cluster and verifying end-to-end telemetry**

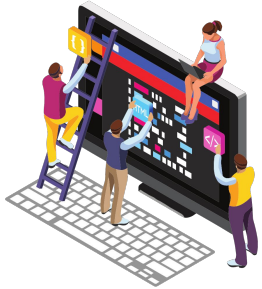
Introduction

Aligning Expectations

Aligning Expectations

What this course is about

Covering the most important aspects of working with OpenTelemetry and instrumenting applications in real-world projects



Gaining hands-on experience through practical labs and projects



Going beyond the basics and covering topics like Manual Instrumentation, Context Propagation, Distributed Tracing, and much more!



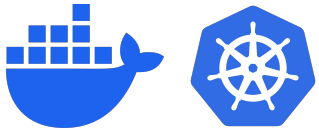
By the end of this course, you will have a strong **practical and theoretical foundation** of both **Observability** and **OpenTelemetry**, and you will be able to take this knowledge and apply it to **real-world, production projects**.

Aligning Expectations

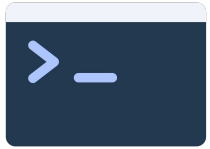
Requirements and prerequisites for making the most of this course



Familiarity with programming.



Familiarity with Docker and Kubernetes.



Familiarity with the terminal / command line.

Observability Foundations

Observability Foundations

Section overview

What Is Observability

1. Understanding system behavior through external signals in distributed architectures.
2. How observability differs from traditional monitoring and why it matters.

The Three Pillars

1. Metrics, logs, and traces as distinct signal types with complementary roles.
2. How combining all three signals provides complete visibility into a system.

Metrics and Histograms

1. Counters, gauges, and histograms — when and why to use each type.
2. Reading percentiles (P50, P95, P99) to understand tail latency and user impact.

Observability Foundations

Section overview

Logs and Traces

1. Structured logging, severity levels, and capturing event context over time.
2. Trace IDs, spans, and parent-child relationships across distributed services.

SLIs, SLOs, and SLAs

1. Defining user-centric reliability targets and aligning them to service commitments.
2. The buffer between internal objectives and external contractual agreements.

Error Budgets and the Four Golden Signals

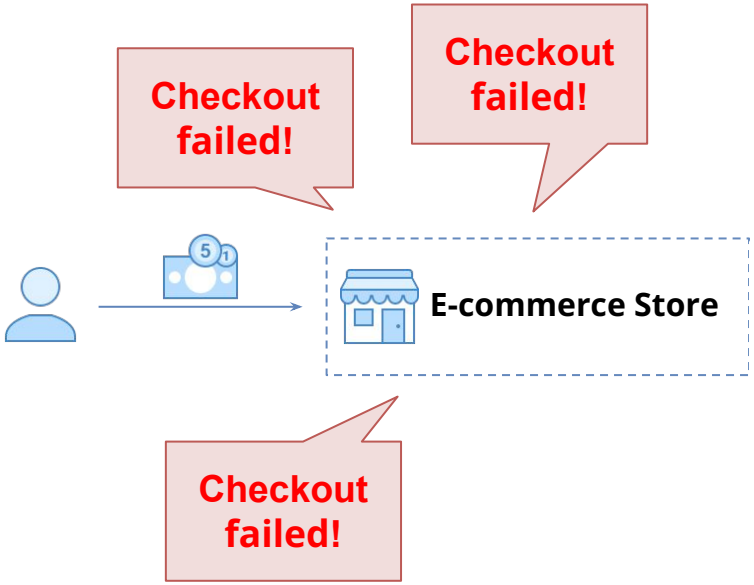
1. Quantifying acceptable failure to balance reliability and development velocity.
2. Latency, traffic, errors, and saturation as a universal monitoring baseline.

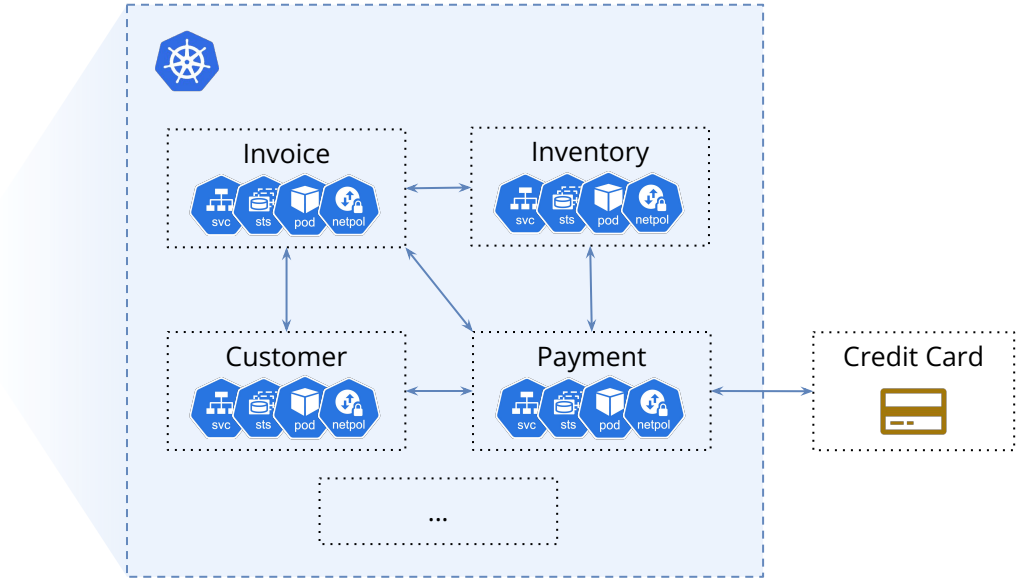
What is Observability?



What is Observability?

Understanding System Behavior Through External Signals





What is Observability?

Understanding System Behavior Through External Signals

Monitoring	Observability
✅ Known questions	✅ Unknown questions
✅ Predefined dashboards	✅ Exploratory investigation
✅ Known failure modes	✅ Unexpected behaviors
✅ "Is the system healthy?"	✅ "Why did this request fail?"

> Alert on `error rate` > 1%

> Why did request `#123` fail at 13:47?

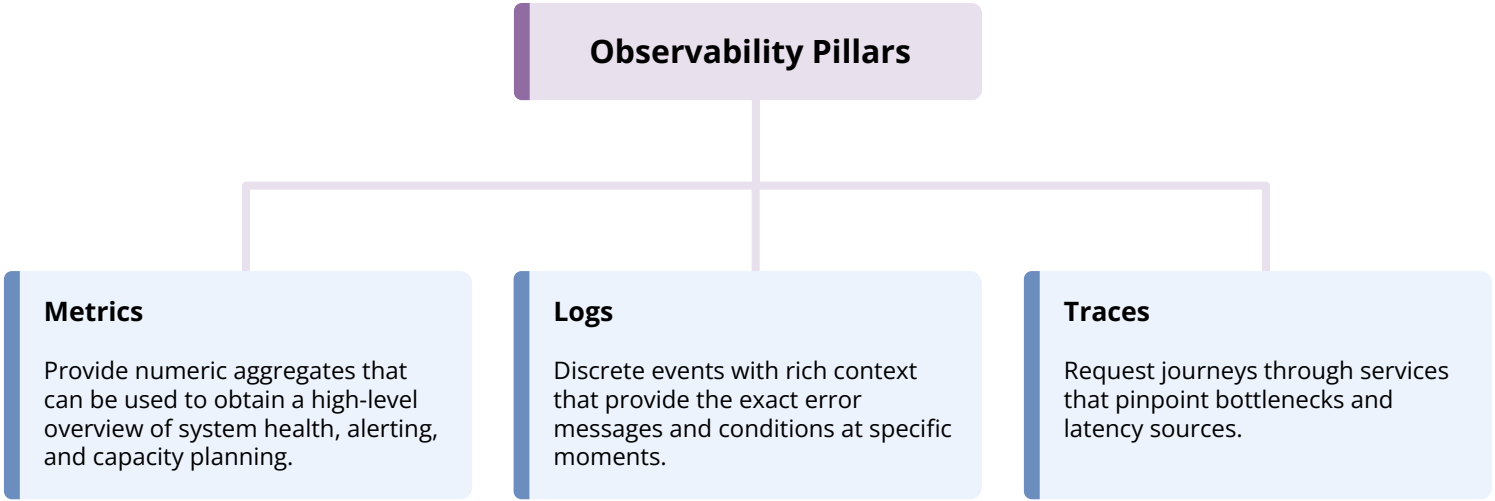
Modern System Complexity

- ✅ Hundreds of microservices vs. Single monolithic applications.
- ✅ Network communication vs. Direct function calls.
- ✅ Dynamic deployment in containers vs. Static servers.
- ✅ Auto-scaling based on demand vs. Peak-load provisioning.
- ✅ Problems emerge from complex interactions between components rather than from single points of failure.

Observability Pillars

Observability Pillars

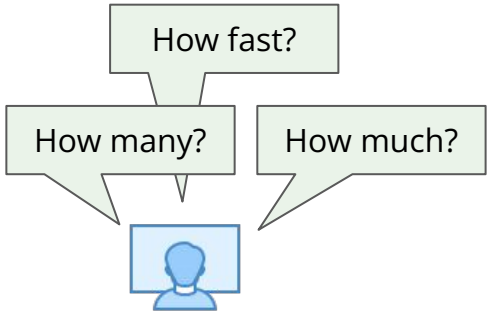
Fundamental Units of Observable Systems



Metrics

Metrics

Quantitative Measurements of a System



Metrics Examples

- ✅ **Request count:** How many HTTP requests?
- ✅ **Error rate:** What percentage of requests are failing?
- ✅ **Latency:** How long do requests take to complete?
- ✅ **CPU usage:** How much CPU are we consuming?
- ✅ **Active connections:** How many database connections are open?

Types of Metrics



Counters

Monotonically increasing values. They only go up. Examples: total HTTP requests, total bytes transferred, total errors



Gauges

Point-in-time measurements that can go up or down. Examples: current memory usage, queue depth, CPU percentage



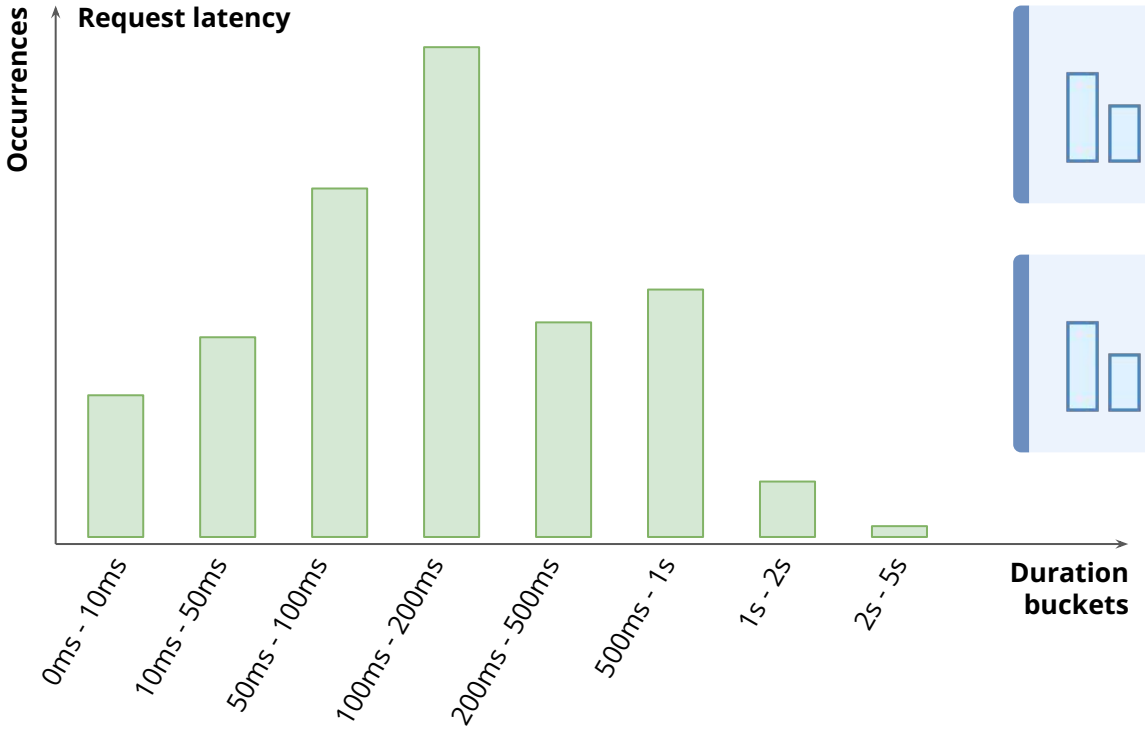
Histograms

Capture distributions of values, providing more insight than just averages. Examples: p95, p99, and p99.9 for request latency.

Histograms

Histograms

Understand How to Read and Interpret Histograms



Histograms

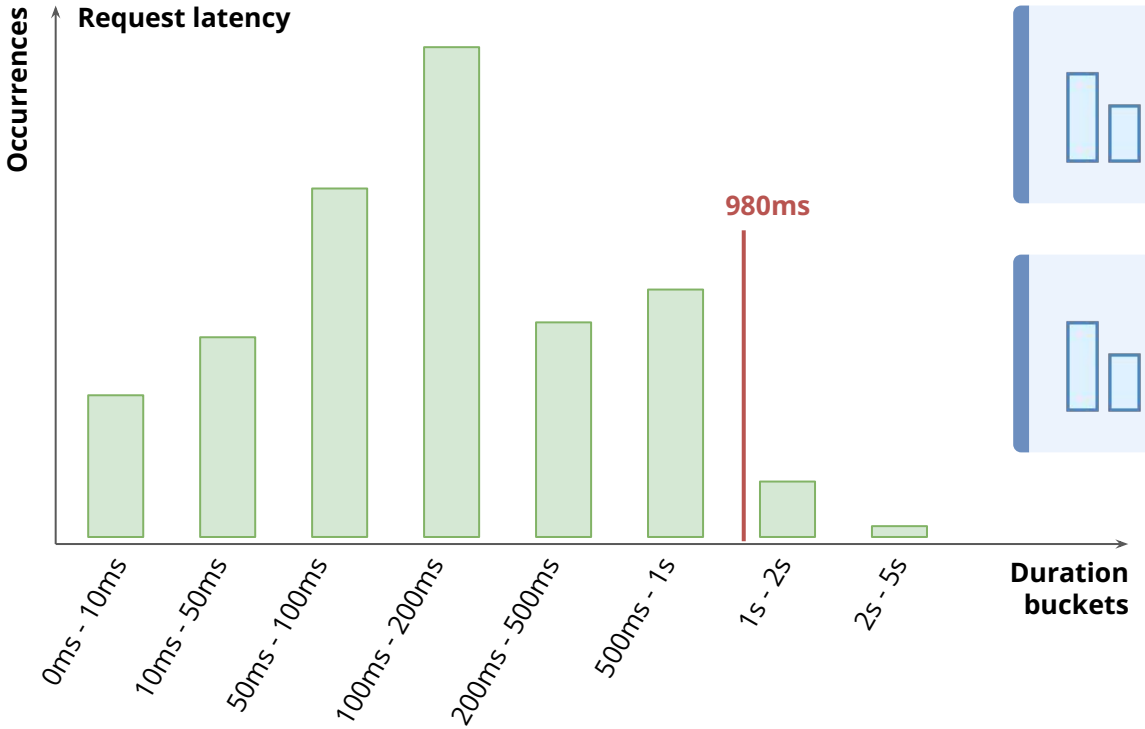
Capture distributions of values, providing more insight than just averages. Examples: p95, p99, and p99.9 for request latency.

Percentiles

The *pX*-Percentile defines a single value *Y* below which *X* percent of the observed values lie.

Histograms

Understand How to Read and Interpret Histograms



Histograms

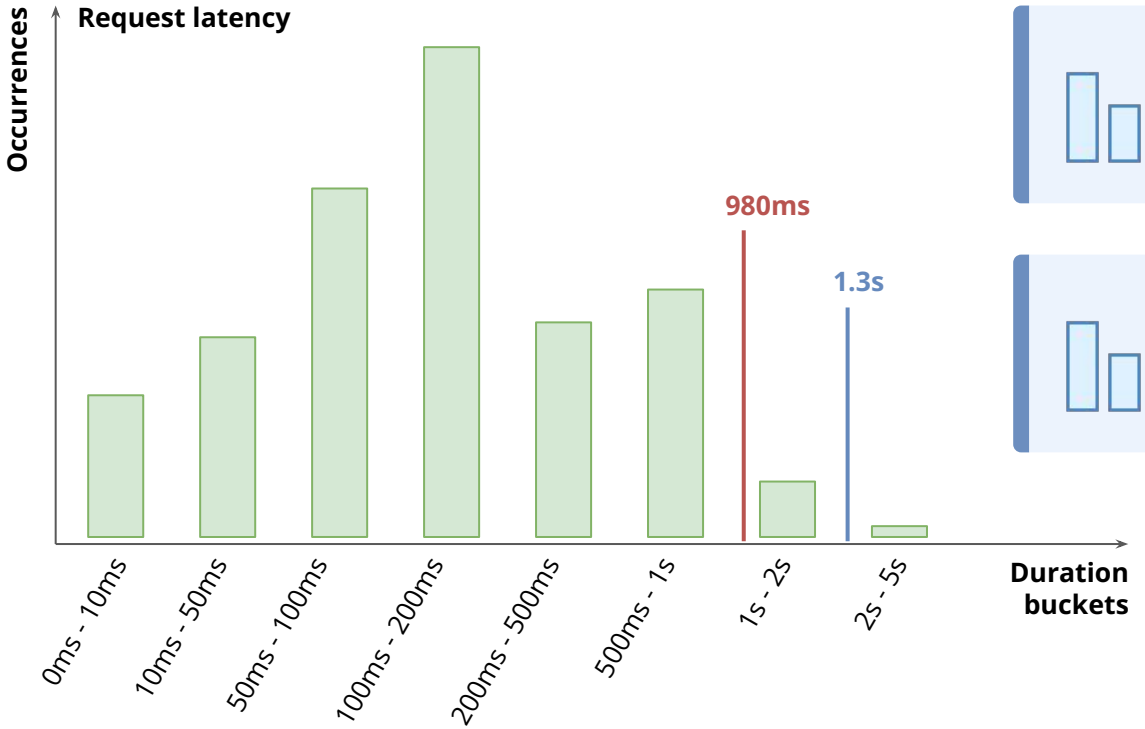
Capture distributions of values, providing more insight than just averages. Examples: **p95**, p99, and p99.9 for request latency.

Percentiles

The *pX*-Percentile defines a single value *Y* below which *X* percent of the observed values lie.

Histograms

Understand How to Read and Interpret Histograms



Histograms

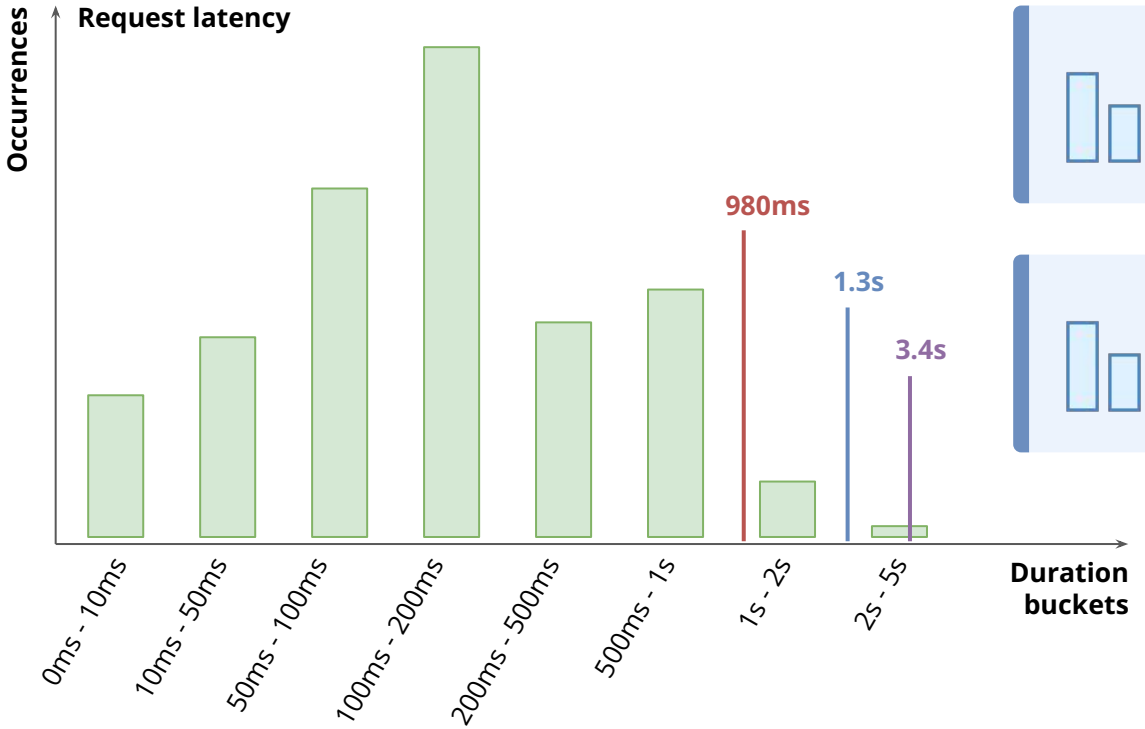
Capture distributions of values, providing more insight than just averages. Examples: **p95**, **p99**, and p99.9 for request latency.

Percentiles

The *pX*-Percentile defines a single value *Y* below which *X* percent of the observed values lie.

Histograms

Understand How to Read and Interpret Histograms



Histograms

Capture distributions of values, providing more insight than just averages. Examples: **p95**, **p99**, and **p99.9** for request latency.

Percentiles

The *pX*-Percentile defines a single value *Y* below which *X* percent of the observed values lie.

Logs

Logs

Discrete Records of Events

- ✅ Capture the **raw details** of discrete events.
- ✅ More expensive to store and index than metrics.
- ✅ Human-readable vs. Machine-readable logs.
- ✅ Multiple levels: `DEBUG`, `INFO`, `WARN`, `ERROR`, `FATAL`.

Human-readable logs

logs

```
2024-01-15 14:32:10 INFO User 12345 clicked checkout
2024-01-15 14:32:11 ERROR Payment gateway timeout: 5000ms
2024-01-15 14:32:11 WARN Retrying payment attempt 2/3
```

Machine-readable (structured) logs

logs

```
{
  "timestamp": "2024-01-15T14:32:10Z",
  "level": "INFO",
  "message": "User clicked checkout",
  "user_id": 12345,
  "cart_items": 3,
  "trace_id": "abc123"
}
```

Logs

Discrete Records of Events

Log Levels in Details

Logs

Discrete Records of Events

Log Levels in Details

✅ **DEBUG:** Detailed information for debugging."Called function X with parameters Y."

Logs

Discrete Records of Events

Log Levels in Details

✅ `DEBUG`**:** Detailed information for debugging."Called function X with parameters Y."

✅ `INFO`**:** Normal operations. "User logged in." "Order processed." Helpful for understanding application flow.

Logs

Discrete Records of Events

Log Levels in Details

- ✅ `DEBUG`: Detailed information for debugging."Called function X with parameters Y."
- ✅ `INFO`: Normal operations. "User logged in." "Order processed." Helpful for understanding application flow.
- ✅ `WARN`: Something unexpected but not an error. "Rate limit approaching." "Cache miss." Potential issues to investigate.

Logs

Discrete Records of Events

Log Levels in Details

- ✅ `DEBUG`**:** Detailed information for debugging."Called function X with parameters Y."
- ✅ `INFO`**:** Normal operations. "User logged in." "Order processed." Helpful for understanding application flow.
- ✅ `WARN`**:** Something unexpected but not an error. "Rate limit approaching." "Cache miss." Potential issues to investigate.
- ✅ `ERROR`**:** Something failed. "Payment gateway timeout." "Database connection refused." Requires attention.

Logs

Discrete Records of Events

Log Levels in Details

- ✅ `DEBUG`**:** Detailed information for debugging."Called function X with parameters Y."
- ✅ `INFO`**:** Normal operations. "User logged in." "Order processed." Helpful for understanding application flow.
- ✅ `WARN`**:** Something unexpected but not an error. "Rate limit approaching." "Cache miss." Potential issues to investigate.
- ✅ `ERROR`**:** Something failed. "Payment gateway timeout." "Database connection refused." Requires attention.
- ✅ `FATAL`**:** Critical failure, application can't continue. "Out of memory." "Config file missing."

Logs

Discrete Records of Events

Log Levels in Details

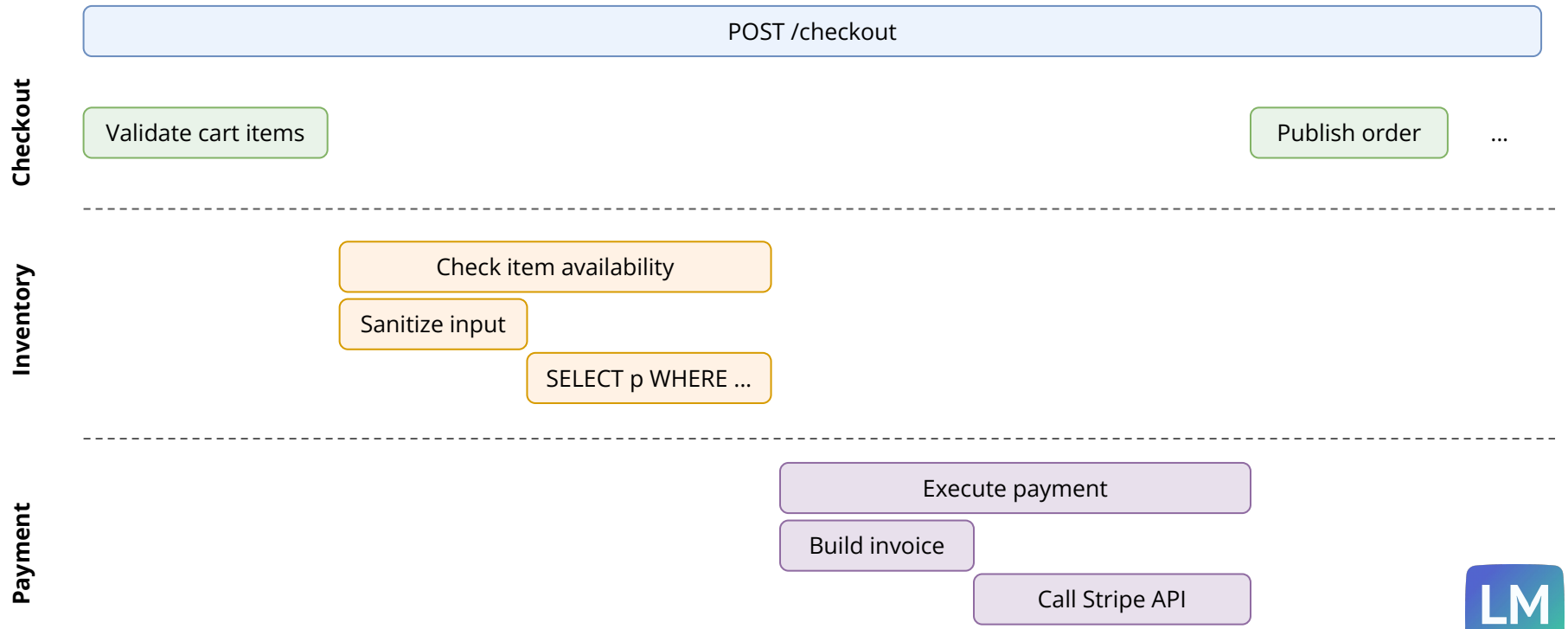
- ✅ **`DEBUG`:** Detailed information for debugging."Called function X with parameters Y."
- ✅ **`INFO`:** Normal operations. "User logged in." "Order processed." Helpful for understanding application flow.
- ✅ **`WARN`:** Something unexpected but not an error. "Rate limit approaching." "Cache miss." Potential issues to investigate.
- ✅ **`ERROR`:** Something failed. "Payment gateway timeout." "Database connection refused." Requires attention.
- ✅ **`FATAL`:** Critical failure, application can't continue. "Out of memory." "Config file missing."

In production we normally enable only `INFO` and higher levels.

Traces

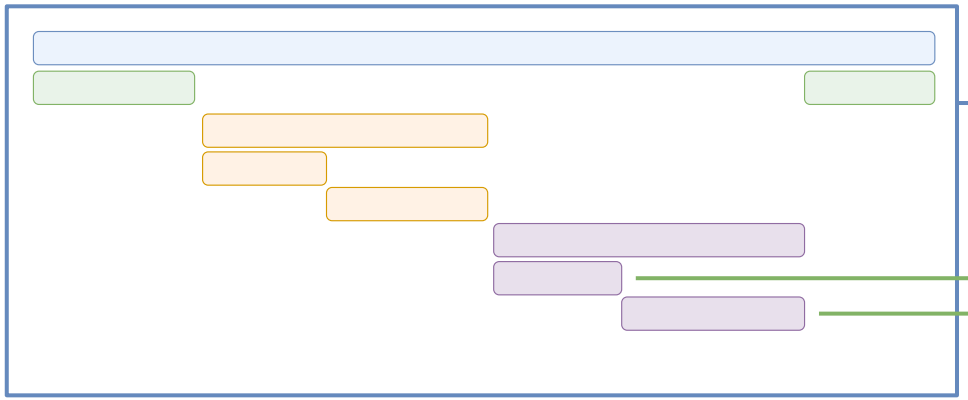
Traces

The Complete Journey of a Request through the System



Traces

The Complete Journey of a Request through the System



Trace

The complete story of one request. It has a unique trace ID that follows the request everywhere it goes.

Span

A single operation within a trace. Each service handling the request creates one or more spans.

Span elements

- ✅ A **name** (like "GET /api/users")
- ✅ **Start** and **end timestamps**
- ✅ **Attributes** (key-value pairs, context)
- ✅ A **parent span ID** (relationships)

Information

- ✅ The complete path the request took
- ✅ How long each operation took
- ✅ Parallel vs. sequential operations
- ✅ Where errors occurred

Which Signals to Use?

Which Signals to Use?

How the Three Signals Complement Each Other

Metrics

- ✅ Dashboards showing system health
- ✅ Alerts on abnormal conditions
- ✅ Capacity planning and trending
- ✅ Aggregated views across instances

Logs

- ✅ Detailed error messages
- ✅ Business logic debugging
- ✅ Audit trails
- ✅ Raw values and rich context

Traces

- ✅ Debugging slow requests
- ✅ Service dependencies
- ✅ Finding where errors originate
- ✅ Optimizing critical paths

Which Signals to Use?

How the Three Signals Complement Each Other

Metrics

- ✅ Dashboards showing system health
- ✅ Alerts on abnormal conditions
- ✅ Capacity planning and trending
- ✅ Aggregated views across instances

Logs

- ✅ Detailed error messages
- ✅ Business logic debugging
- ✅ Audit trails
- ✅ Raw values and rich context

Traces

- ✅ Debugging slow requests
- ✅ Service dependencies
- ✅ Finding where errors originate
- ✅ Optimizing critical paths

1. Metrics alert you
    a. Your alert fires: **HTTP error rate > 1% for 5 minutes.**

Which Signals to Use?

How the Three Signals Complement Each Other

Metrics

- ✅ Dashboards showing system health
- ✅ Alerts on abnormal conditions
- ✅ Capacity planning and trending
- ✅ Aggregated views across instances

Logs

- ✅ Detailed error messages
- ✅ Business logic debugging
- ✅ Audit trails
- ✅ Raw values and rich context

Traces

- ✅ Debugging slow requests
- ✅ Service dependencies
- ✅ Finding where errors originate
- ✅ Optimizing critical paths

1. Metrics alert you
   a. Your alert fires: **HTTP error rate > 1% for 5 minutes.**
2. Traces show you where
   a. You look for traces with **HTTP status 500** and inspect the **trace structure** of failing requests.

Which Signals to Use?

How the Three Signals Complement Each Other

Metrics

- ✅ Dashboards showing system health
- ✅ Alerts on abnormal conditions
- ✅ Capacity planning and trending
- ✅ Aggregated views across instances

Logs

- ✅ Detailed error messages
- ✅ Business logic debugging
- ✅ Audit trails
- ✅ Raw values and rich context

Traces

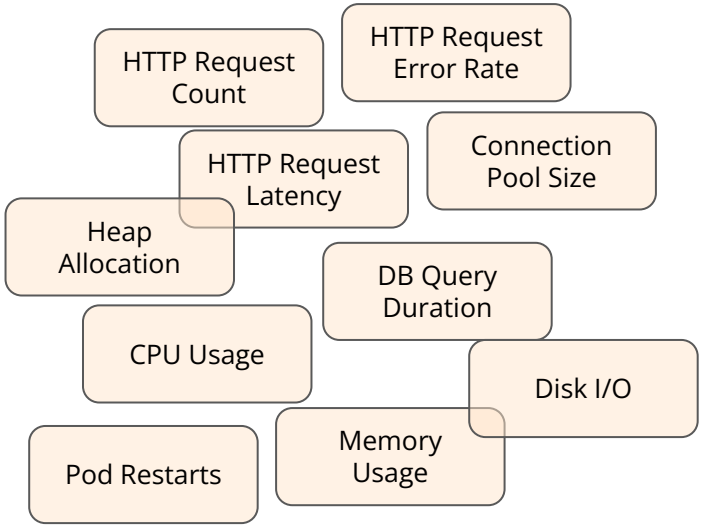
- ✅ Debugging slow requests
- ✅ Service dependencies
- ✅ Finding where errors originate
- ✅ Optimizing critical paths

1. Metrics alert you
   a. Your alert fires: **HTTP error rate > 1% for 5 minutes.**
2. Traces show you where
   a. You look for traces with **HTTP status 500** and inspect the **trace structure** of failing requests.
3. Logs explain why
   a. You filter your logs by the trace ID from one failing request, finding the exact error: *Payment gateway returned 503 Service Unavailable - rate limit exceeded.*

Service Level Indicators (SLIs)

Service Level Indicators (SLIs)

What to Measure?



Service Level Indicators (SLIs)

A Service Level Indicator is a carefully selected metric that **directly measures** a service's behavior **from the user's perspective**.

Examples of Good SLIs

API Svc

- ✅ **Availability:** Percentage of non-5xx responses
- ✅ **Latency:** Percentage of completion within 500ms

Batch jobs

- ✅ **Throughput:** Percentage of timely completion
- ✅ **Correctness:** Percentage of correct completion

DB

- ✅ **Availability:** Percentage of queries that succeed
- ✅ **Latency:** Percentage of completion within 100ms

Service Level Objectives (SLOs)

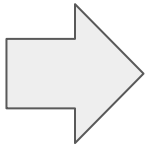
Service Level Objectives (SLOs)

Set Specific Target Values for SLIs

> **Service Level Objectives (SLOs)**
>
> An SLO is a target value for an SLI. It's the promise you make about service quality.

Examples of SLIs

- ✅ **Availability:** Percentage of non-5xx responses
- ✅ **Latency:** Percentage of completion within 500ms
- ✅ **Throughput:** Percentage of timely completion
- ✅ **Correctness:** Percentage of correct completion



Examples of SLOs

- ✅ **99.95% of requests** return non-5xx status codes
- ✅ **99% of requests** complete within 500ms
- ✅ **99.5% of reports** generate within 1 hour of data arrival
- ✅ **99.99% of calculations** match validation checks

Service Level Agreements (SLAs)

Service Level Agreements (SLAs)

Contractual Agreements with Customers

Service Level Agreements (SLAs)

The contractual commitment you make with your customers regarding a certain SLI. SLAs should always be less than SLOs.

Why Not Promise More?

Why not promise 99.99% or 99.999% availability?

The answer is **cost** and **opportunity cost**.

Each additional nine of availability requires **exponentially more effort**.

How It Probably Would Look Like:

✅ **99%** (about 7 hours downtime per month)
Relatively Easy

✅ **99.9%** (about 43 minutes per month)
Requires good practices

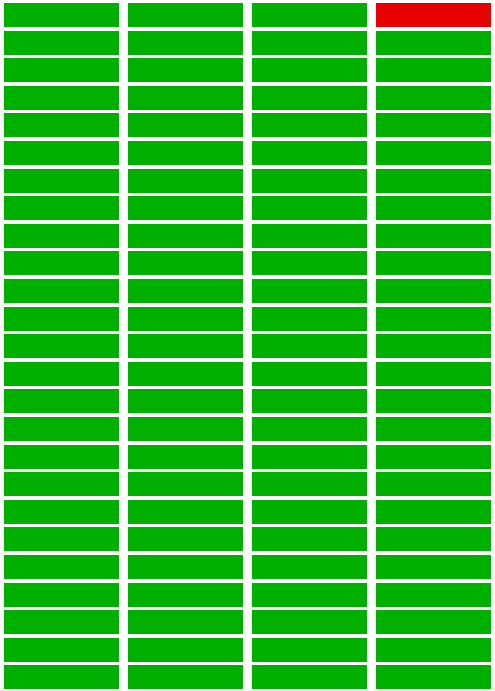
✅ **99.99%** (about 4 minutes per month)
Requires sophisticated automation

✅ **99.999%** (about 25 seconds per month)
Requires massive investment

Error Budgets

Error Budgets

How Much You Can Afford to Fail



Error Budget

The error budget is the difference between 100% and the set SLO. If your SLO is 99.9% availability, your error budget is 0.1% failure. Over a 30-day month:

- ✅ Total minutes: 43,200
- ✅ Allowed downtime: 43.2 minutes
- ✅ That's your **error budget**

Error Budget > 0?

- ✅ Push risky features faster
- ✅ Deploy more frequently
- ✅ Run experiments in production
- ✅ Take calculated risks

Error Budget <= 0?

- ✅ Freeze risky deployments
- ✅ Focus on reliability improvements
- ✅ Pay down technical debt
- ✅ Investigate and fix the root causes of recent outages

The Four Golden Signals

The Four Golden Signals

Google's Four Metrics that Apply to Almost Every Service

Latency: How long does it take?

- ✅ Measure response time for requests
- ✅ Distinguish between successful request latency and failed request latency
- ✅ Track percentiles (p50, p95, p99) not just averages.

Traffic: How much demand?

- ✅ Measure requests per second, queries per second, or whatever units make sense for your system.
- ✅ Helps normalize other metrics and understand system capacity.

Errors: What's the rate of failures?

- ✅ Measure the percentage of requests that fail.
- ✅ Define errors explicitly (are 4xx responses also errors? Are timeouts errors?)

Saturation: How "full" is the system?

- ✅ Measure resource utilization: CPU, memory, disk, network bandwidth, database connections.
- ✅ High saturation indicates you're approaching limits and should expect degraded performance soon.

Setting Up the Observability Stack

Setting Up the Observability Stack

Section overview

Local Environment Setup

Deploy Observability Backends

1. Understand the roles of Prometheus, Loki, Tempo, and Grafana as an observability stack.
2. Deploy each service in our Compose application.

Application and Architecture

1. The distributed translation application as the instrumentation target for the course.
2. How telemetry signals flow from the application through backends into Grafana.

Prometheus in 5 Minutes

Prometheus in 5 Minutes

Understand the Main Components and Characteristics of Prometheus



Prometheus

Time-series database designed for storing metrics.

PromQL: The Query Language

Query language for time-series metrics data, enabling aggregations and mathematical operations over dimensional data

Pull-Based System

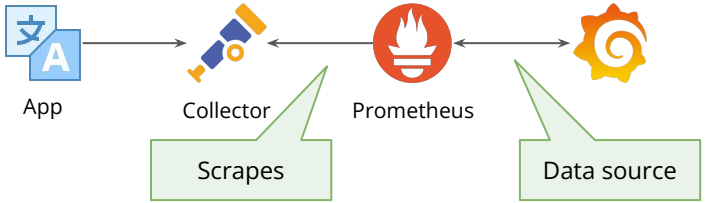


- ✅ **Filter by labels** using curly braces `http_requests_total{status="500"}`
- ✅ **Functions** and **operators** `rate(http_requests_total[5m])`
- ✅ **Aggregators** `sum by (path) (rate(http_requests_total[5m]))`
- ✅ **Percentiles** `histogram_quantile(0.99, ...)`

Data Structure

```
http_requests_total{method="GET", path="/api/users"}

http_requests_total{method="POST", path="/api/users"}
```



Loki in 5 Minutes

Loki in 5 Minutes

Understand the Main Components and Characteristics of Loki

Loki

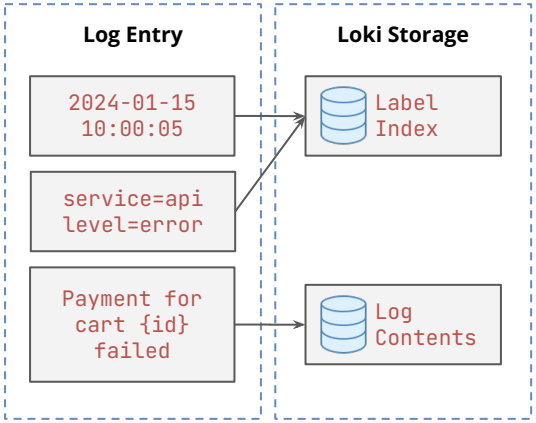
Log aggregation system designed to be cost-effective and scalable.

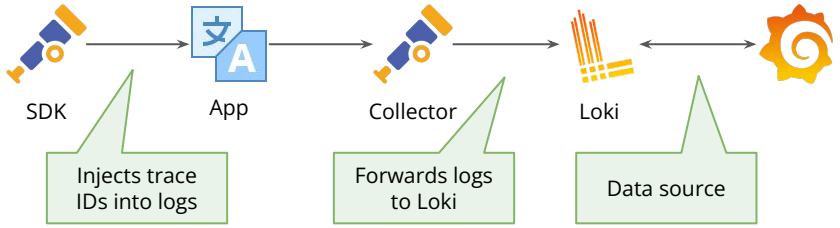
LogQL: The Query Language

Query language for log data that combines label selection with log stream filtering and parsing

- ✅ **Filter by labels** using curly braces `{job="frontend", environment="production"}`
- ✅ **Line filters** for content search `{job="frontend"} |= "error" |~ "timeout|failed"`
- ✅ **Parsers** extract structured data `{job="frontend"} | json | status_code >= 500`
- ✅ **Log rates and volumes** `sum(rate({job="frontend"} |= "error" [5m]))`

Metadata Indexing





Tempo in 5 Minutes

Tempo in 5 Minutes

Understand the Main Components and Characteristics of Tempo



Tempo

Distributed tracing backend designed to store traces cost-effectively at scale.

TraceQL: The Query Language

Query language for distributed traces that enables searching spans by attributes and analyzing trace structure

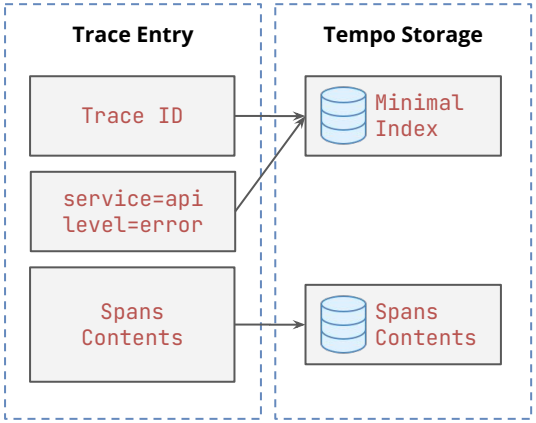
✅ **Filter by attributes** `{service.name="frontend" && http.request.method="GET"}`

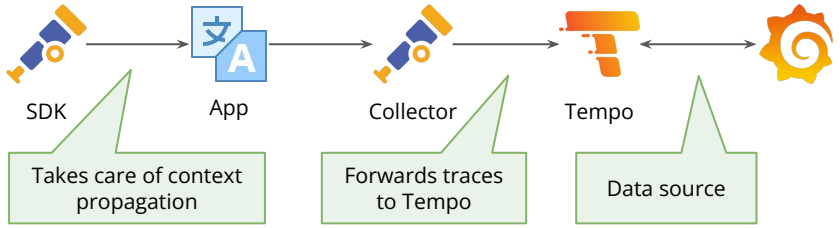
✅ **Duration filtering** `{service.name="frontend" && duration > 100ms}`

✅ **Structural queries** (parent-child) `{span.http.method="GET"} >> {status = error}`

✅ **Aggregations** `count({status = error}) by (resource.service.name)`

Metadata Indexing

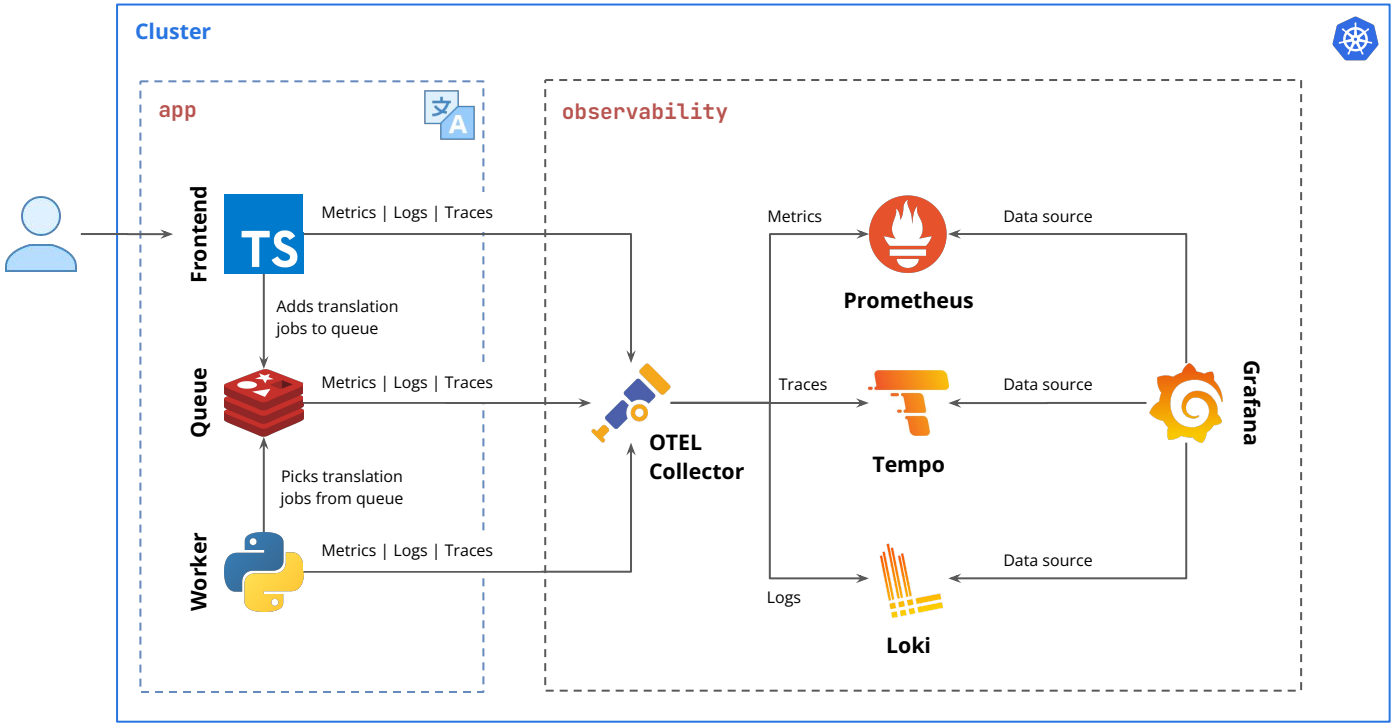




System Architecture

System Architecture

Understand the Application and Observability Stack We Will Work With



Introduction to OpenTelemetry Concepts



Introduction to OpenTelemetry Concepts

Section overview

What is OpenTelemetry and Why It Exists

APIs and SDKs

1. Understand the separation between the two.

Automatic and Manual Instrumentation

1. Auto-instrumentation for out-of-the-box framework coverage with no code changes.
2. Manual instrumentation for capturing custom business logic and domain context.

The OpenTelemetry Collector

Semantic Conventions

What Is OpenTelemetry?

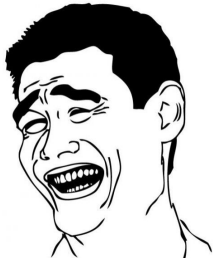


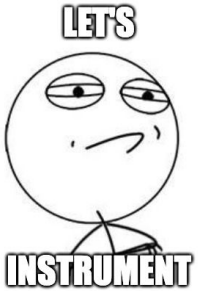
What Is OpenTelemetry?

Understand What OpenTelemetry Is and Why It Exists











What Is OpenTelemetry?

Understand What OpenTelemetry Is and Why It Exists

OpenTelemetry

An **observability framework** and **toolkit** designed to **create** and **manage telemetry data** such as traces, metrics, and logs.

❌ It's not yet another backend to store and visualize your data.

✅ It is a standardized, vendor-neutral layer between your application and whatever backend you use.

- **Vendor-neutral APIs** for creating telemetry data. You call OpenTelemetry APIs, not vendor-specific ones.
- **SDKs** that implement those APIs and handle the complexity of batching, sampling, and exporting data.
- **Automatic instrumentation libraries** for popular frameworks and languages.
- **OpenTelemetry Collector:** a vendor-agnostic pipeline that can receive, process, and export telemetry data.

What Is OpenTelemetry?

Understand What OpenTelemetry Is and Why It Exists

OpenTelemetry

An **observability framework** and **toolkit** designed to **create** and **manage telemetry data** such as traces, metrics, and logs.

✅ Vendor Neutrality

You instrument once and can switch backends without changing your code. Want to evaluate three different tracing backends? No problemo!

✅ Industry standardization

OpenTelemetry is backed by all major observability vendors. Prometheus, Grafana, DataDog, New Relic - they all support OpenTelemetry.

✅ Rich ecosystem

Because OpenTelemetry is a standard, there's a huge library of automatic instrumentation. Most of your telemetry can be generated automatically!

OpenTelemetry APIs and SDKs



OpenTelemetry APIs and SDKs

Understand the Difference Between OpenTelemetry APIs and SDKs

OpenTelemetry API

Collection of interfaces and abstract types that define what a `Span` is, what a `Metric` is, and how to create them.

Frameworks like Flask, FastAPI, Express, should depend on the API.

OpenTelemetry SDKs

The actual implementation of those interfaces. It's the code that creates real spans, batches them, samples traces, etc.

The application team should configure the SDK in the application code.



```python
class Span(abc.ABC):                                                    [docs]
    """A span represents a single operation within a trace."""


    @abc.abstractmethod                                                 [docs]
    def get_span_context(self) -> "SpanContext":
        """Gets the span's SpanContext.

        Get an immutable, serializable identifier for this span that can be
        used to create new child spans.

        Returns:
            A :class:`opentelemetry.trace.SpanContext` with a copy of this span's immutable st
        """


    @abc.abstractmethod                                                 [docs]
    def set_attributes(
        self, attributes: typing.Mapping[str, types.AttributeValue]
    ) -> None:
        """Sets Attributes.

        Sets Attributes with the key and value passed as arguments dict.

        Note: The behavior of `None` value attributes is undefined, and hence
        strongly discouraged. It is also preferred to set attributes at span
```

```python
class Span(trace_api.Span, ReadableSpan):                               [docs]
    """See `opentelemetry.trace.Span`.

    Users should create `Span` objects via the `Tracer` instead of this
    constructor.

    Args:
        name: The name of the operation this span represents
        context: The immutable span context
        parent: This span's parent's `opentelemetry.trace.SpanContext`, or
            None if this is a root span
```

```python
    def get_span_context(self):                                         [docs]
        return self._context


    def set_attributes(                                                 [docs]
        self, attributes: Mapping[str, types.AttributeValue]
    ) -> None:
        with self._lock:
            if self._end_time is not None:
                logger.warning("Setting attribute on ended span.")
                return

            for key, value in attributes.items():
                self._attributes[key] = value
```



OpenTelemetry APIs and SDKs

Understand the Difference Between OpenTelemetry APIs and SDKs

instrumentation.ts

```ts
// In instrumentation.ts - SDK configuration
import { NodeSDK } from '@opentelemetry/sdk-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-o

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: 'http://otel-collector:4318/v1/traces',
  }),
  serviceName: 'translator-api',
});

sdk.start();
```

OpenTelemetry APIs and SDKs

Understand the Difference Between OpenTelemetry APIs and SDKs

instrumentation.ts

```ts
// In instrumentation.ts - SDK configuration
import { NodeSDK } from '@opentelemetry/sdk-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-o

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: 'http://otel-collector:4318/v1/traces',
  }),
  serviceName: 'translator-api',
});

sdk.start();
```

> The SDK configuration happens once, in the application code during startup.

OpenTelemetry APIs and SDKs

Understand the Difference Between OpenTelemetry APIs and SDKs

instrumentation.ts

```ts
// In instrumentation.ts - SDK configuration
import { NodeSDK } from '@opentelemetry/sdk-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-o

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: 'http://otel-collector:4318/v1/traces',
  }),
  serviceName: 'translator-api',
});

sdk.start();
```

app.ts

```ts
// In app.ts - API usage
import { trace } from '@opentelemetry/api';

const tracer = trace.getTracer('translator');

app.post('/', (req, res) => {
  const span = tracer.startSpan('translate_content');
  // ... business logic ...
  span.end();
});
```

OpenTelemetry APIs and SDKs

Understand the Difference Between OpenTelemetry APIs and SDKs

instrumentation.ts

```ts
// In instrumentation.ts - SDK configuration
import { NodeSDK } from '@opentelemetry/sdk-node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-o

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({
    url: 'http://otel-collector:4318/v1/traces',
  }),
  serviceName: 'translator-api',
});

sdk.start();
```

> The rest of our application code (and the underlying framework) depends on the API, not directly on the SDK.

app.ts

```ts
// In app.ts - API usage
import { trace } from '@opentelemetry/api';

const tracer = trace.getTracer('translator');

app.post('/', (req, res) => {
  const span = tracer.startSpan('translate_content');
  // ... business logic ...
  span.end();
});
```

OpenTelemetry APIs and SDKs

Understand the Difference Between OpenTelemetry APIs and SDKs

OpenTelemetry SDKs

When you configure the OpenTelemetry SDK in your application, what are you actually specifying?

- **Exporters:** Where should telemetry go? OTLP to a Collector? Directly to Prometheus? To the console for debugging?
- **Processors:** How should telemetry be processed? Batch spans for efficiency? Sample only a percentage of traces? Add custom attributes?
- **Samplers:** Which traces should be recorded? Sample 100% for development? 10% for production? Use probabilistic sampling?
- **Resource attributes:** What metadata should be attached? Service name, version, deployment environment?

OpenTelemetry Instrumentation

OpenTelemetry Instrumentation

Understand the Differences Between Automatic and Manual Instrumentation

Automatic Instrumentation

- ✅ You get telemetry for frameworks and libraries without writing any code.
- ✅ OpenTelemetry provides instrumentation libraries for popular frameworks.



Manual Instrumentation

- ✅ Explicitly write code to create spans, add attributes, and record metrics.
- ✅ **Why?** Automatic instrumentation doesn't know about your business logic. It knows that you called a database, but it doesn't know what business operation that database call was part of.

Spans that represent business logic:

```
HTTP POST /images
upload_article_images
```

automatic

```
1 Span: BLPOP translation:jobs
2   - db.system: redis
3   - db.operation: BLPOP
```

manual

```
1 Span: process_translation_job
2   - translation.job_id: abc-123
3   - translation.target_language: es
4   - translation.status: success
```

Which One to Use?

- ✅ That depends! Normally start with auto-instrumentation: **many metrics with little effort**
- ✅ Once instrumented, extend the code base to include manual instrumentation for relevant business operations.

The OpenTelemetry Collector

The OpenTelemetry Collector

Understand What the Collector Does and How It Works

The OpenTelemetry Collector

Vendor-agnostic telemetry data pipeline that can receive telemetry data in multiple formats, process it in various ways, and export it to one or more backends.

Deployment Patterns

Agent Deployment

Run a Collector instance on every host or pod alongside your applications.

Gateway Deployment

Run the Collector as a centralized service that multiple applications send to.



Source: https://opentelemetry.io/docs/collector/

The OpenTelemetry Collector

Understand What the Collector Does and How It Works

The OpenTelemetry Collector

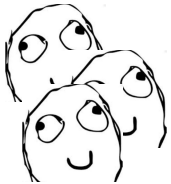
Should I / When to use the Collector? There are some pretty good reasons and use cases:

- **Decouple backends from application code:** The Collector configuration is separate from your app code, so you can change exporters independently.
- **Send telemetry to multiple backends simultaneously:** Maybe you're evaluating tools, or you want both an internal Prometheus and an external vendor.
- **You want centralized sampling or filtering:** Rather than configuring sampling in every application, configure it once in the Collector.
- **Legacy migrations:** The Collector can receive Jaeger or Zipkin traces and forward them as OTLP, letting you migrate incrementally.
- **Transform or enrich data:** Add attributes, scrub sensitive data, or reshape telemetry before it reaches backends.

Semantic Conventions

Semantic Conventions

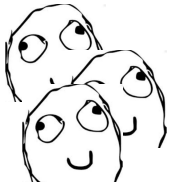
Standardized Naming Rules for Spans, Metrics, and Attributes



Team NodeJS



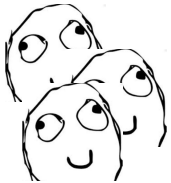
```
http.method
http.statusCode
```



Team Python



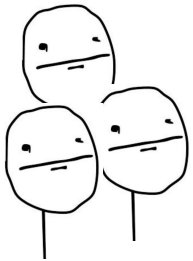
```
request.method
response.status
```



Team Go



```
httpMethod
status_code
```



Team SRE

> Show me all HTTP requests that returned 500 errors.

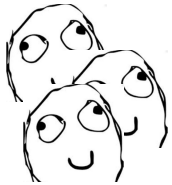
Semantic Conventions

Standardized Naming Rules for Spans, Metrics, and Attributes

Semantic Conventions

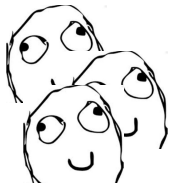
They define a standard vocabulary for telemetry data. Everyone follows them, independently of language or framework.

- **Span names:** What should a span be called?
- **Attribute names:** What should attributes be called? Not `db_name` or `databaseName` but `db.name`.
- **Attribute values:** What values are allowed? Status codes are integers, HTTP methods are uppercase (`GET`, `POST`).
- **Required vs. Optional:** Which attributes must always be present versus which are nice-to-have?



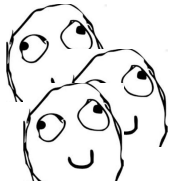
Team NodeJS

```
http.request.method
http.response.status_code
```



Team Python

```
http.request.method
http.response.status_code
```



Team Go

```
http.request.method
http.response.status_code
```

Semantic Conventions

Standardized Naming Rules for Spans, Metrics, and Attributes

app.ts

```ts
import {
  SEMATTRS_HTTP_METHOD,
  SEMATTRS_HTTP_STATUS_CODE,
} from '@opentelemetry/semantic-conventions';

span.setAttribute(SEMATTRS_HTTP_METHOD, 'GET');
span.setAttribute(SEMATTRS_HTTP_STATUS_CODE, 200);
```

app.py

```python
from opentelemetry.semconv.trace import SpanAttributes

span.set_attribute(SpanAttributes.HTTP_METHOD, "GET")
span.set_attribute(SpanAttributes.HTTP_STATUS_CODE, 200)
```

Advantages of Using Constants

- ✅ You can't typo the attribute name
- ✅ Auto-complete helps you discover available conventions
- ✅ Your code stays up-to-date if convention names evolve
- ✅ It's self-documenting: readers know you're following standards

You can also create your own attributes!

app.ts

```ts
span.setAttribute('myapp.user.tier', 'premium');
span.setAttribute('myapp.feature.flag.new_ui', true);
span.setAttribute('myapp.experiment.variant', 'B');
```

Instrumenting Single Services



Instrumenting Single Services

Section overview

Setting Up the OpenTelemetry Collector

1. Deploying the Collector as a vendor-agnostic pipeline between the application and observability backends.
2. Verifying end-to-end data flow and monitoring the Collector's own operational health.

Instrumenting Metrics

1. Applying automatic instrumentation for framework-level metrics and creating custom business metrics.
2. Enriching metrics with dimensional labels and exploring them in Prometheus.

Instrumenting Traces

1. Capturing framework spans automatically and creating custom spans for business logic.
2. Managing span lifecycle, handling errors, and refactoring instrumentation for maintainability.

Instrumenting Single Services

Section overview

Instrumenting Logs

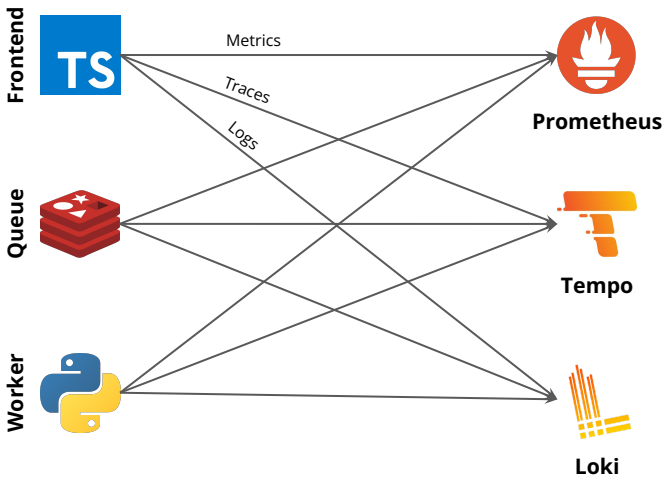
1. Configuring structured logging with trace context injection and adding manual log statements.
2. Setting up log-trace correlation in Grafana and externalizing configuration through environment variables.

OpenTelemetry Collector

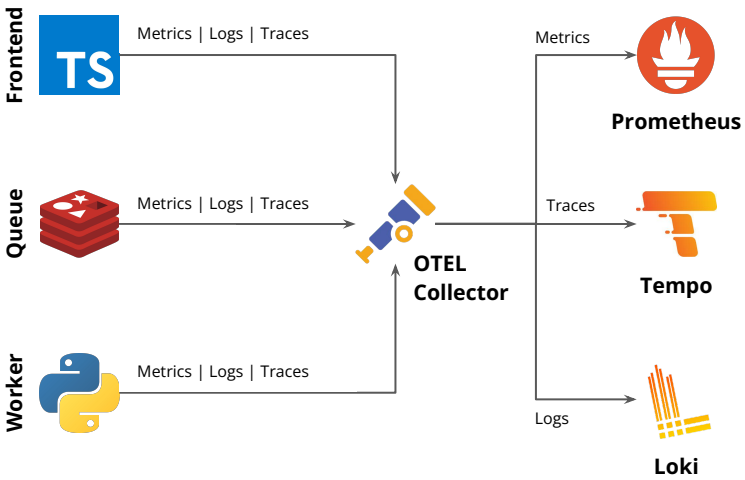
OpenTelemetry Collector

Explore the Use Case and Architecture of the OpenTelemetry Collector

Without a Collector



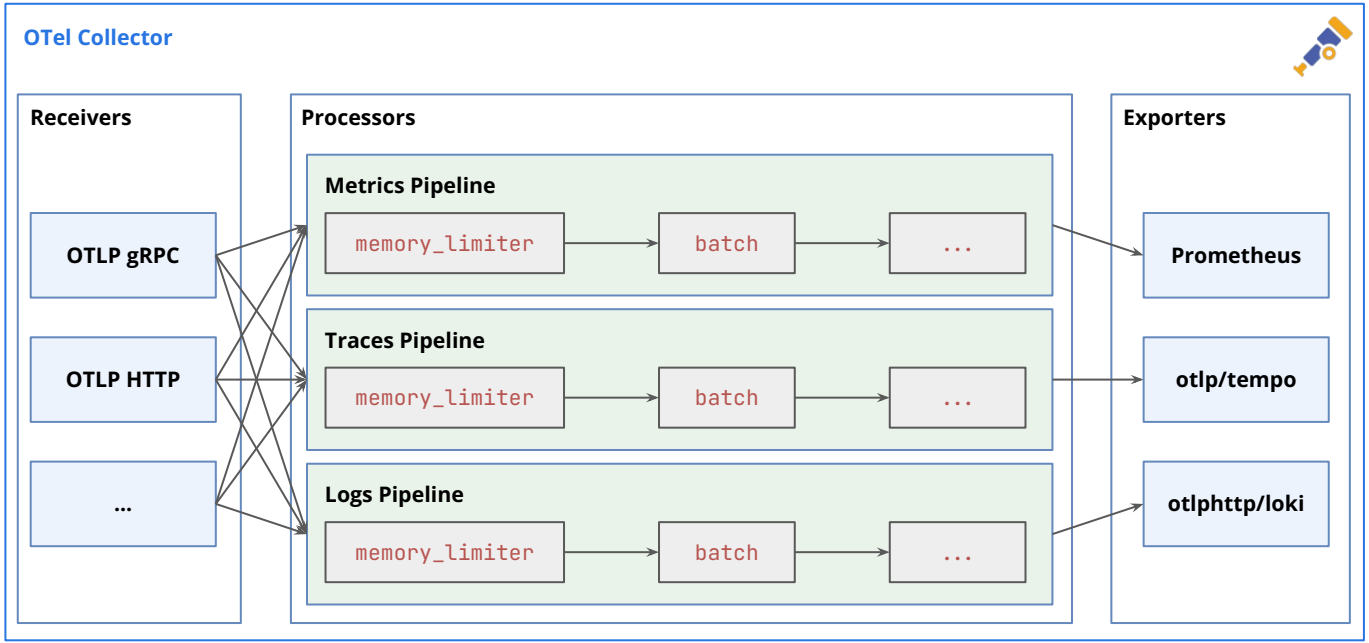
With a Collector



OpenTelemetry Collector

Explore the Use Case and Architecture of the OpenTelemetry Collector

The Collector's Architecture



otel-config.yaml

```yaml
processors:
  memory_limiter:
    check_interval: 1s
    limit_mib: 512
    spike_limit_mib: 128
  batch:
    timeout: 10s
    send_batch_size: 1024
    send_batch_max_size: 2048
```

A few other processors:

- ✅ attributes
- ✅ filter
- ✅ tail_sampling
- ✅ k8sattributes
- ✅ transform

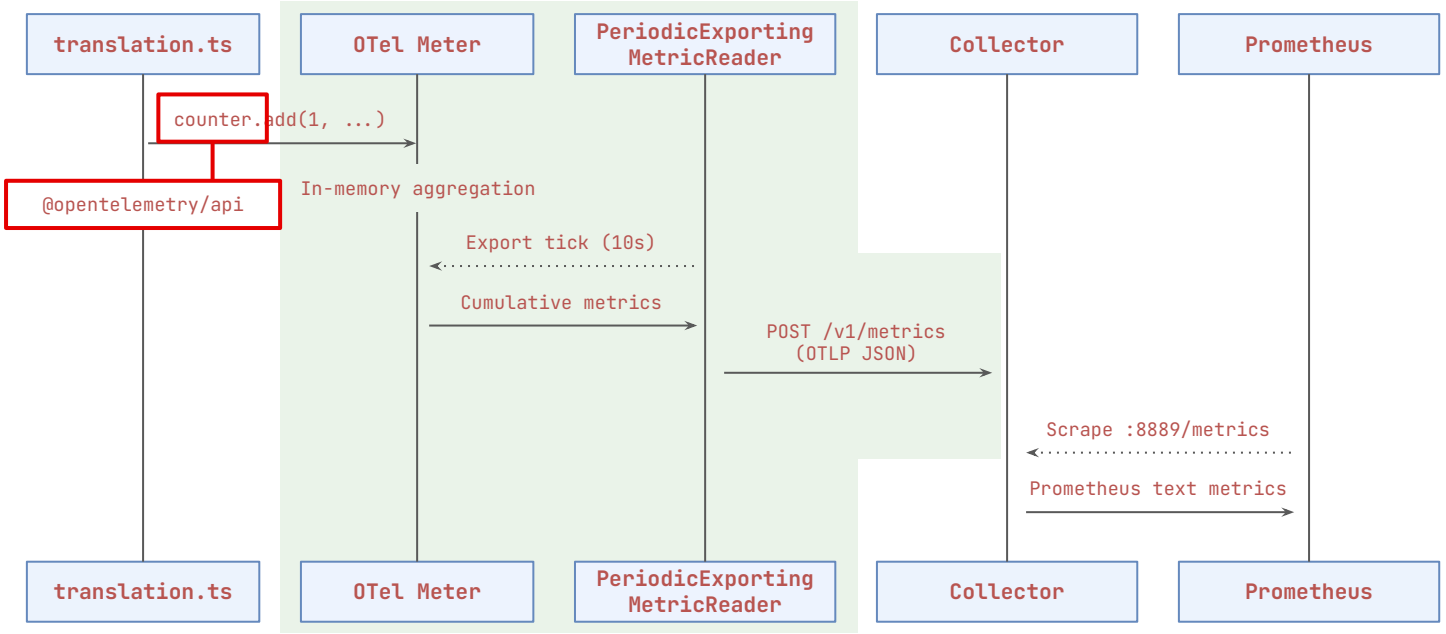
Metrics Instrumentation



Metrics Instrumentation

Understand the Role of the OpenTelemetry SDK and Instrumentation Techniques

End-to-End Data Flow



Metrics Instrumentation

Understand the Role of the OpenTelemetry SDK and Instrumentation Techniques

instrumentation.ts

```ts
const sdk = new NodeSDK({
  serviceName: 'translation-frontend',
  metricReader: new PeriodicExportingMetricReader({
    exporter: new OTLPMetricExporter({
      url: 'http://otel-collector:4318/v1/metrics',
    }),
    exportIntervalMillis: 10000, // push every 10 s
  }),
  instrumentations: [getNodeAutoInstrumentations()]
});
```

Auto-Instrumentation

- ✅ `http_server_duration_milliseconds_count{...}`
- ✅ `nodejs_eventloop_utilization_ratio{...}`
- ✅ `...`

Manual Instrumentation

metrics.ts

```ts
import { metrics } from '@opentelemetry/api';

const meter = metrics.getMeter('translation-frontend', '1.0.0');

export const translationRequestsCounter = meter.createCounter(
  'translation.requests.total',
  { description: 'Total translation requests' },
);
```

translation.ts

```ts
import { translationRequestsCounter } from '../metrics';

router.post('/', async (req, res) => {
  ...

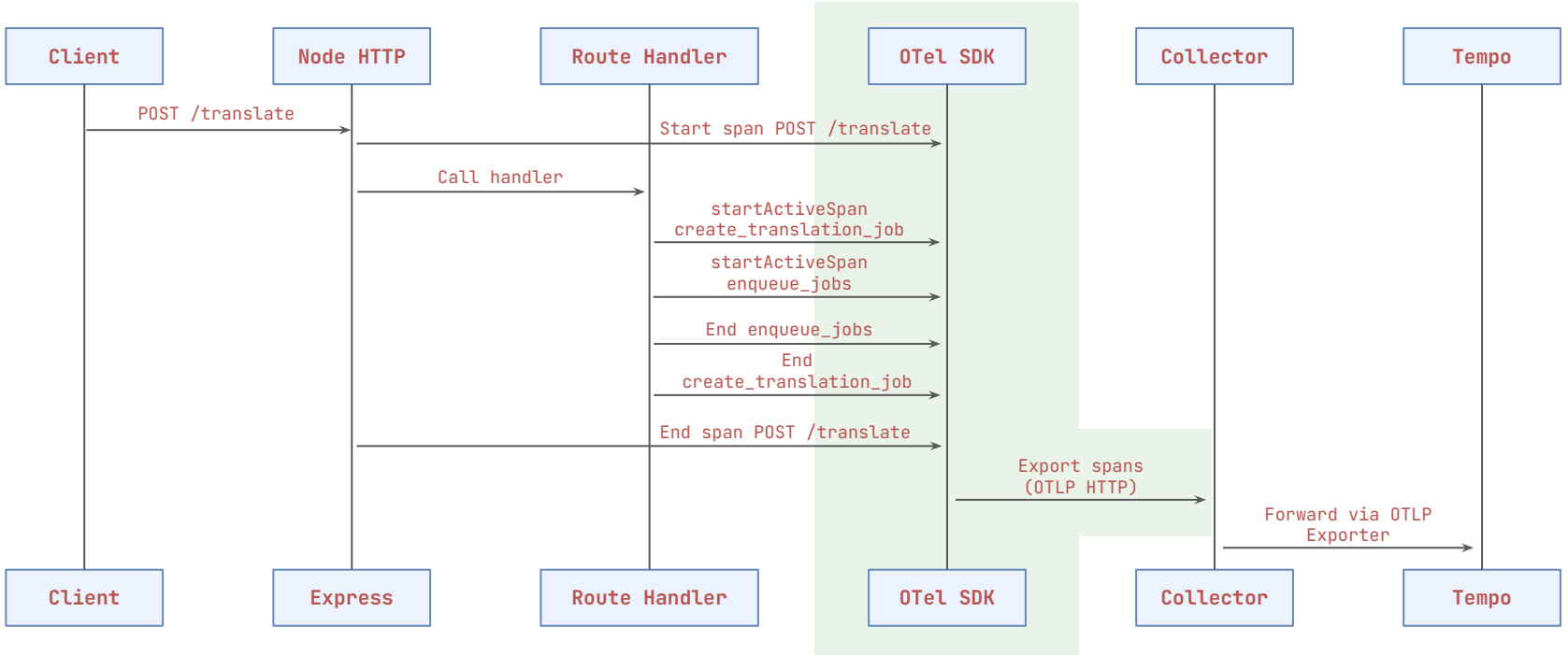
  translationRequestsCounter.add(1, {
    status: 'success',
    language_count: targetLanguages.length.toString(),
  });
```

Traces Instrumentation

Traces Instrumentation

Understand the Role of the OpenTelemetry SDK and Instrumentation Techniques

End-to-End Data Flow



Traces Instrumentation

Understand the Role of the OpenTelemetry SDK and Instrumentation Techniques

instrumentation.ts

```ts
const sdk = new NodeSDK({
  ...
  traceExporter: new OTLPTraceExporter({
    url: 'http://otel-collector:4318/v1/traces', // ← new
  }),
  ...
  instrumentations: [getNodeAutoInstrumentations()],
});

sdk.start();
```

Manual Instrumentation

✅ Calling `startActiveSpan` automatically sets the new span as a child of the currently active span, so no need to pass context manually.

Manual Instrumentation

translation.ts

```ts
// src/routes/translation.ts
const tracer = trace.getTracer('translation-frontend', '1.0.0');

router.post('/', async (req, res) => {
  return tracer.startActiveSpan('<span_name>', async (span) => {
    try {
      // business logic here
      span.setAttribute('translation.text_length', body.text.length);
      span.setAttribute('translation.target_languages', langs.join(','));

      // ... do work ...

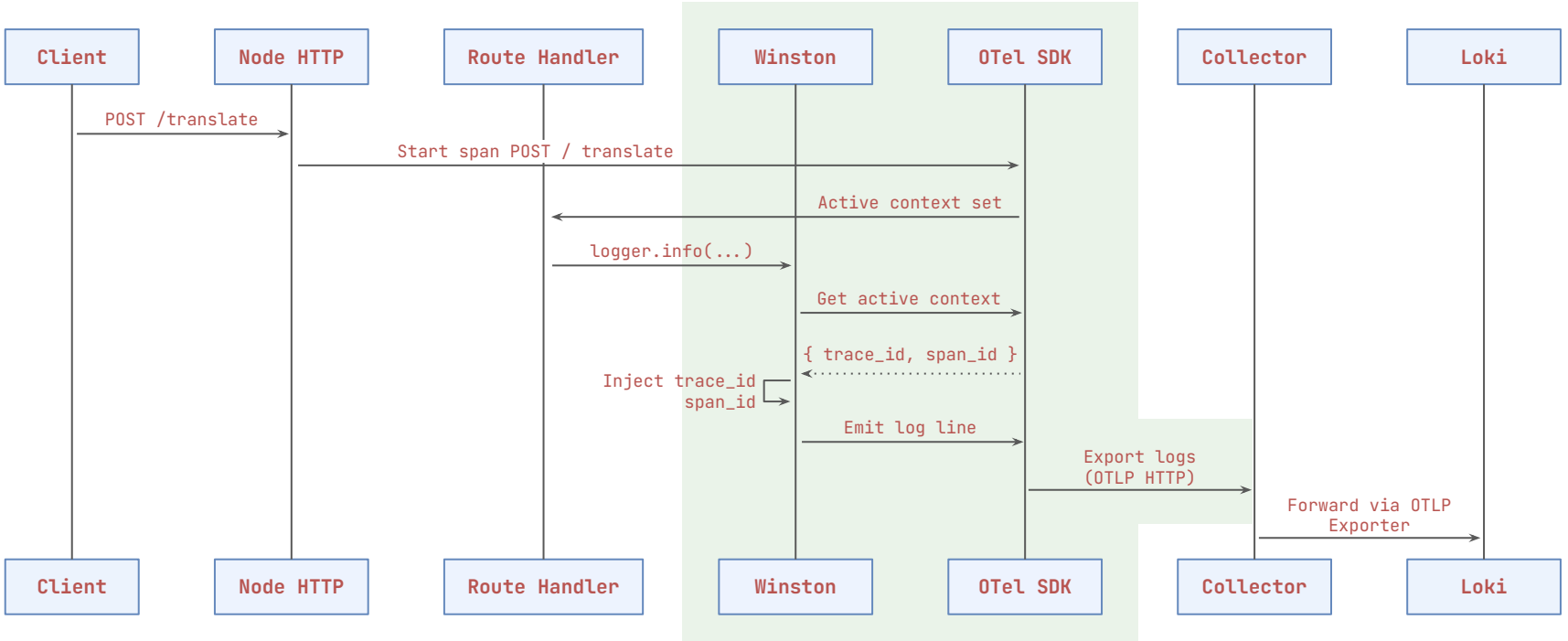
      span.setStatus({ code: SpanStatusCode.OK });
    } catch (err) {
      span.recordException(err as Error);
      span.setStatus({
        code: SpanStatusCode.ERROR,
        message: String(err)
      });
      throw err;
    } finally {
      span.end(); // always end the span
    }
  });
});
```

Logs Instrumentation

Logs Instrumentation

Understand the Role of the OpenTelemetry SDK and Instrumentation Techniques

End-to-End Data Flow



Logs Instrumentation

Understand the Role of the OpenTelemetry SDK and Instrumentation Techniques

instrumentation.ts

```ts
const sdk = new NodeSDK({
  ...
  logRecordProcessors: [
    new BatchLogRecordProcessor(
      new OTLPLogExporter({
        url: 'http://otel-collector:4318/v1/logs'
      })
    )
  ],
  instrumentations: [
    ...
    new WinstonInstrumentation(),
  ],
});

sdk.start();
```

Manual Instrumentation

logger.ts

```ts
import winston from 'winston';

export const logger = winston.createLogger({
  level: process.env.LOG_LEVEL?.toLowerCase() || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json(),
  ),
  transports: [new winston.transports.Console()],
});
```

translation.ts

```ts
import { logger } from '../logger';

// Validation failure
logger.warn('Validation failed: too many languages', {
  reason: 'too_many_languages',
  requested_count: body.targetLanguages.length,
  max_allowed: 3,
});
```

Instrumenting Distributed Systems



Instrumenting Distributed Systems

Section overview

Auto-Instrumenting the Worker

1. Collecting metrics, logs, and traces automatically from the Python worker service.
2. Observing that worker spans are isolated from frontend traces and understanding why.

Distributed Context Propagation

1. Injecting and extracting trace context across an asynchronous message queue boundary.
2. Completing bidirectional propagation to unify frontend and worker spans into a single trace.

Advanced Worker Instrumentation

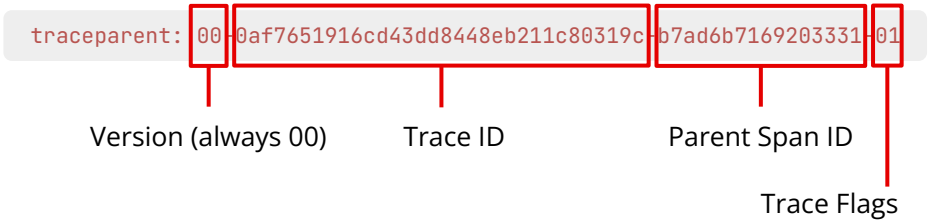
1. Adding custom metrics and spans, monitoring Redis with the exporter pattern, and externalizing configuration.
2. Migrating worker logs to structured format for consistent cross-service querying and correlation.

Context Propagation

Context Propagation

Understand How Automatic and Manual Context Propagation Work

Automatic Context Propagation



Fields:

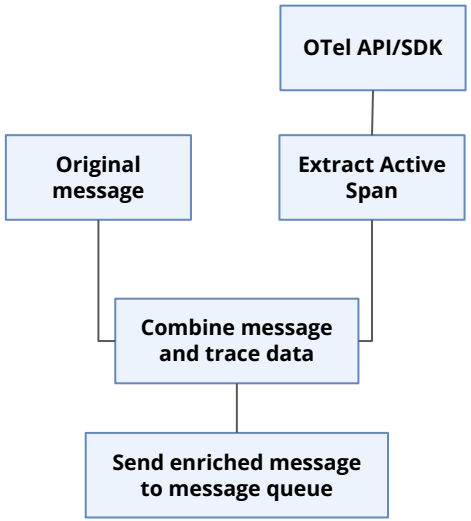
- ✅ `version`: Always 00 (current version)
- ✅ `trace-id`: Unique ID for the entire distributed trace (128-bit)
- ✅ `parent-span-id`: ID of the parent span (64-bit)
- ✅ `trace-flags`: Sampling decision and other flags (8-bit)

Automatic Propagation:

- ✅ `HTTP`: Done automatically via the traceparent header
- ✅ `gRPC`: Done automatically via metadata

Manual Context Propagation:

- ✅ Message queues don't have HTTP headers
- ✅ The trace context must be embedded in the message payload



Context Propagation

Understand How Automatic and Manual Context Propagation Work

Manual Context Propagation

queue.ts

```ts
import { propagation, context } from '@opentelemetry/api';

// ...

const _traceContext: Record<string, string> = {};
propagation.inject(context.active(), _traceContext);

const jobJson = JSON.stringify({
    ...job,
    _traceContext
});
await this.client.lpush(this.queueKey, jobJson);

// ...
```

worker.py

```python
from opentelemetry import trace, propagate
from opentelemetry.trace import SpanKind, StatusCode

# ...

remote_ctx = propagate.extract(job.get("_traceContext", {}))

with tracer.start_as_current_span(
    "process_translation_job",
    context=remote_ctx,
    kind=SpanKind.CONSUMER,
    attributes={
        "translation.job_id": job_id,
        "translation.session_id": session_id,
        "translation.target_language": target_lang,
        "translation.text_length": len(text) if text else 0,
    },
) as span:
# ...
```

Deploying to Kubernetes



Deploying to Kubernetes

Section overview

Application Kubernetes Manifests

1. Migrating translation application services from Docker to Kubernetes resources.
2. Applying resource allocation and service exposure best practices for each component.

Observability Stack Manifests

1. Creating Kubernetes resources for the OTel Collector, Prometheus, Loki, Tempo, and Grafana.
2. Adding Kube State Metrics with the RBAC permissions needed for cluster visibility.

Kustomize Configuration

1. Organizing manifests with Kustomize for consistent, reproducible deployments.
2. Automating resource generation and pinning image versions across namespaces.



Deploying to Kubernetes

Section overview

Deploying and Verifying Telemetry End to End

1. Applying the full stack to a local Kubernetes cluster and monitoring startup across namespaces.
2. Confirming that metrics, logs, and traces flow correctly through the Kubernetes stack.